

Features the Developers Could Not Test

The parts of SD Core for Windows that were built and reasoned about but never exercised, what is known about each, and what it would take to settle it.

Everything else in this documentation describes behaviour that was run and watched. **This page is the exception, and it exists so that the exception is visible in one place rather than scattered through the reference as footnotes.**

Nothing here is known to be broken. Each entry is something that **compiles, or exists, or follows from the source, and was never put under load or into the condition that would prove it.** Treat them as the list of things to pilot before an application depends on them.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

Why it is in the administrator set. A reference page that keeps saying "*this part was not tested*" teaches an application programmer to distrust the whole book. An administrator deciding what to put into production needs exactly that information. So it is gathered here, and the user-facing pages state what SD does without qualifying every other paragraph.

How to read an entry

Known	what was actually run and observed
Not known	the specific gap — usually narrower than the heading suggests
To settle it	what would have to be done

Sessions and terminals

INTERACTIVE SD OVER SSH, AT A REAL TERMINAL

Known. An ssh session lands inside SD, and SD's terminal layer was watched driving a real Windows console.

Not known. Those two at once. Nobody has run an interactive session at a terminal *reached over ssh*, where the pseudo-terminal belongs to the ssh server rather than to the Windows console host. Screen handling, cursor positioning and the editing keys all go through that layer.

To settle it. One interactive session from a second machine, driving a full-screen operation and the arrow keys.

Locking and contention

SEMAPHORES UNDER CONTENTION

Known. The semaphores are exercised on every record lock, and two sessions competing for the same record ran through them at once without misbehaving.

Not known. No semaphore has ever been observed blocking. What is unmeasured is the waiting path, not the code path — the difference matters only under a load heavier than anything yet run.

To settle it. Enough concurrent sessions to make one wait, and a watch on what it does while it waits.

CONTENTION BETWEEN AN API SESSION AND A LOCAL ONE

Known. Two *local* sessions compete correctly: record, update and file locks are all reported against the right holder, a waiting read is released when the holder lets go, and a task lock refuses a second taker.

Not known. The same contest with one side arriving through the API server rather than at a terminal.

To settle it. An API client and a terminal session competing for one record.

TASK LOCKS TAKEN TWICE BY THE SAME SESSION

Known. From the source: taking a task lock you already hold succeeds, and one `unlock` releases it however many times you locked it — the ownership test is "*unowned or mine*", and `unlock` clears the slot outright.

Not known. It was **read rather than run**. No program has taken the same task lock twice and then released it once.

To settle it. Four lines of SD BASIC.

Application data

A REAL APPLICATION'S DATA

Known. SD creates, writes, reads and deletes files, records, indexes, select lists and sequential files, and the system files it bootstraps with are real ones.

Not known. No production application's data has been loaded into this port. Nothing here has met a file of hundreds of thousands of records, a deep dictionary, or a schema built over years by somebody else.

To settle it. Restore an existing account and run it.

Sockets

UDP AND ICMP

Known. TCP works: listening, connecting, accepting, reading, writing, the blocking and non-blocking modes, and the error codes.

Not known. The `0x00010000` and `0x00020000` flags are named in the documentation because they are **in the compiler**, not because a datagram was ever sent. No UDP or ICMP socket has been opened.

To settle it. A datagram to a listener and back.

Scheduled tasks

TASK SCHEDULER WITH AN ACCOUNT THAT `CREATE.ACCOUNT` MADE

Known. Scheduled SD jobs work when the task runs as a Windows account you already had. That is the case the design was built around.

Not known. Accounts that `create.account` creates are **denied interactive logon at this machine on purpose**, and whether Task Scheduler will accept one of them as the identity a task runs as has never been tried. It may refuse the credential outright.

To settle it. Create an account, point a scheduled task at it, and see whether the task page accepts it.

SD BASIC statements that compile but were never run

WHY NOT

<code>sendmail</code>	needs a mail relay configured
<code>chgphant()</code>	needs a phantom process to change
<code>ccall()</code>	needs a C function registered into the executable

Known. All three compile in an ordinary account.

Not known. What any of them does. Nothing else in the documentation depends on them.

Third-party editors

The key bindings documented for Microsoft Edit are the editor's own, read from its source rather than driven at a keyboard. SD installs it and calls it; it does not implement it. If a binding differs from what is written, the editor is right and the page is wrong.

What is NOT on this page, and why

Anything that was tested and failed is a defect, not a gap, and does not belong here — it is either fixed or it is a known issue.

Anything a reader might merely find surprising is not a gap either. The places where this port deliberately differs from OpenQM, ScarletDME or SD on Linux are documented as differences, in the pages that describe the feature. This page is only about what nobody has watched happen.

See also

Sessions and Locks covers the locking model that two of the entries above qualify. Installation and the service covers scheduled jobs and the account model behind the Task Scheduler entry. SD System Limits states which of its figures come from the source and which from a running system, and is the other page in this set that distinguishes the two.
